6.2

Regressão linear pela equação normal

Do gráfico de pontos até a sua própria classe MinimosQuadrados, com inv, solve e pinv.

$$\hat{\beta} = (X^T X)^{-1} X^T y$$

Teoria · Exemplos resolvidos · NumPy · Desafio

O caminho da aula

A

Entendendo o problema

Reta, resíduo e mínimos quadrados

B

Da reta para as matrizes

X, y, β e a coluna de 1s

C

Dedução da equação normal

Do erro à fórmula, passo a passo

D

Inversa, solve e pinv

Três jeitos de resolver e quando falham

E

Python com NumPy

California Housing e scikit-learn

F

Erros comuns

O que quebra e como corrigir

G

Desafio final

fit, predict e score

META Ao chegar no bloco G, você já terá usado cada peça da classe MinimosQuadrados pelo menos uma vez.

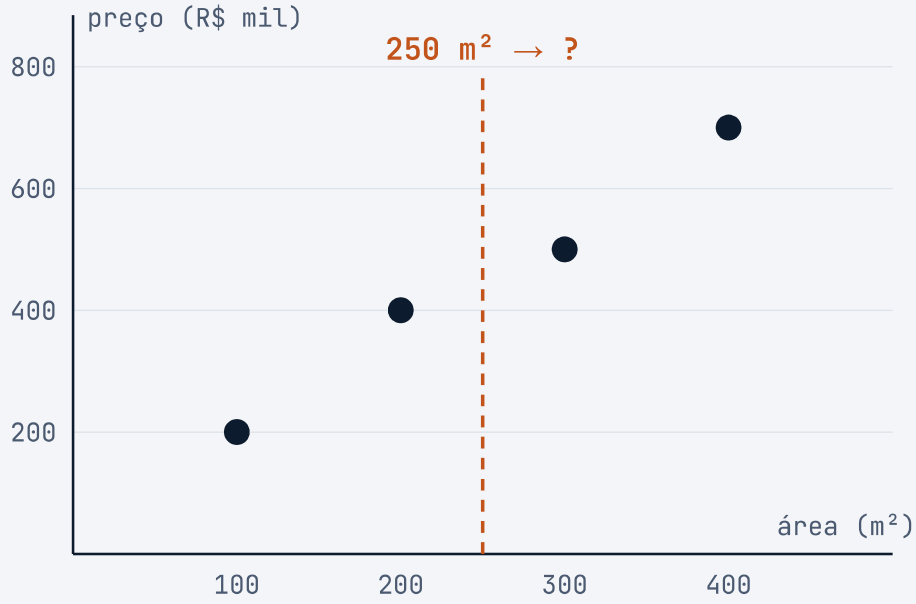
PROBLEMA REAL

Quanto vale uma casa de 250 m²?

Temos o preço de quatro casas já vendidas. Queremos uma regra que, dada a área, estime o preço de qualquer outra.

casa	área (m²)	preço (R$ mil)	x	y
1	100	200	1	2
2	200	400	2	4
3	300	500	3	5
4	400	700	4	7

Para as contas ficarem simples: x = área em centenas de m², y = preço em R$ 100 mil.



INTUIÇÃO

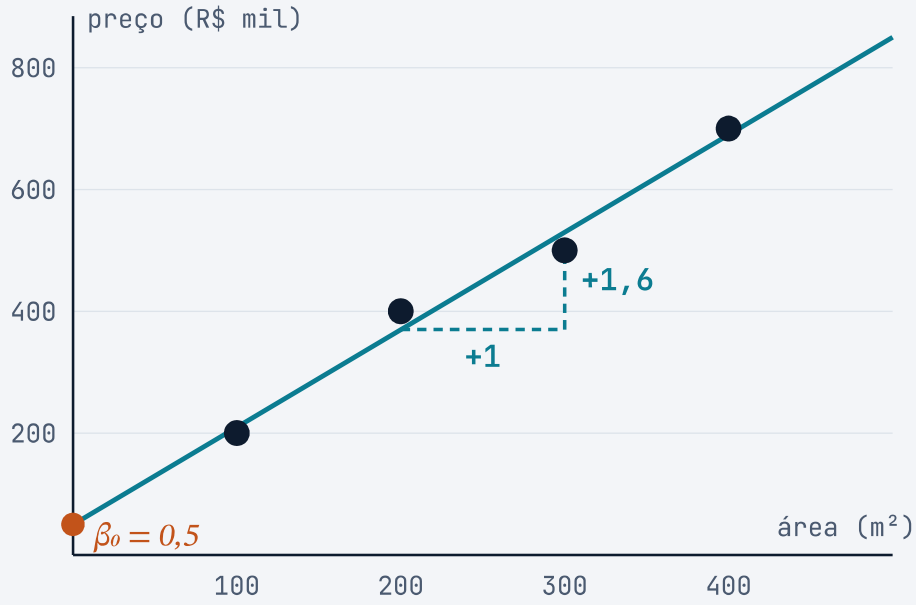
Uma reta é uma regra de previsão

$$\hat{y} = \beta_0 + \beta_1 \cdot x$$

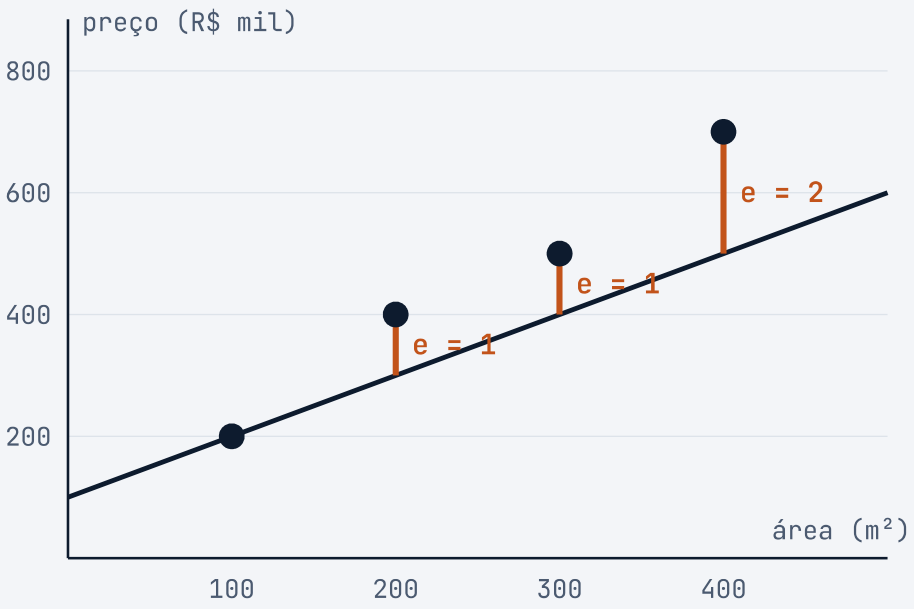
símbolo	significado
$\hat{y}$	predição: o preço que a reta estima
β_0	intercepto: valor de ŷ quando x = 0
β_1	inclinação: quanto ŷ sobe quando x aumenta 1

EXEMPLO

Com $\beta_0 = 0{,}5$ e $\beta_1 = 1{,}6$, uma casa de 250 m² (x = 2,5) recebe $\hat{y} = 0{,}5 + 1{,}6 \cdot 2{,}5 = 4{,}5$, ou seja, R$ 450 mil.



Uma reta qualquer erra. Quanto?



DEFINIÇÃO

Resíduo: $e_i = y_i - \hat{y}_i$

A distância vertical entre o ponto real e a reta.
Positivo quando o ponto está acima.

x	y	ŷ = 1 + x	e = y - ŷ
1	2	2	0
2	4	3	1
3	5	4	1
4	7	5	**2**

Esta reta subestima o preço de quase todas as casas.

Por que somar os erros ao quadrado

reta	resíduos	Σ e	Σ e²
ŷ = 4,5 (horizontal)	−2,5 −0,5 0,5 2,5	**0**	13
ŷ = 1 + x	0 1 1 2	4	6
ŷ = 0,5 + 1,6x	−0,1 0,3 −0,3 0,1	**0**	**0,2**

Sem cancelamento

A soma simples diz que a reta horizontal é perfeita. Ao quadrado, todo erro conta como positivo.

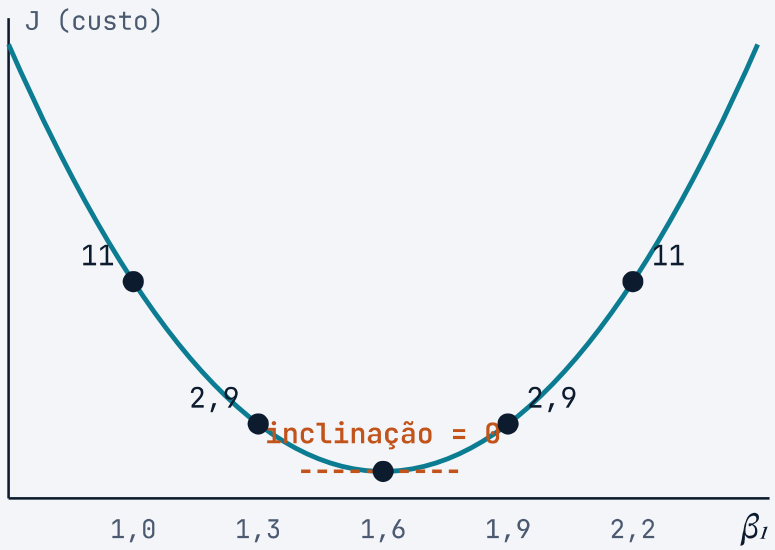
Erros grandes pesam mais

Errar 1 custa 1; errar 2 custa 4. A reta evita erros grandes.

Curva lisa

O custo vira uma curva suave, e o mínimo pode ser encontrado com derivada.

DEFINIÇÃO Mínimos quadrados: escolher β_0 e β_1 que tornam $J = \Sigma\,(y_i - \hat{y}_i)^2$ o menor possível.



EXEMPLO VISUAL

Cada reta tem um custo

Fixando $\beta_0 = 0{,}5$ e variando β_1, calculamos J para cada reta. O resultado é uma parábola.

Com β_0 e β_1 variando juntos, a curva vira uma tigela. O fundo continua plano em todas as direções.

CONCLUSÃO No fundo da curva a inclinação é zero. Essa condição vai dar a fórmula de β.

PERGUNTA RÁPIDA · 2 MIN

Teste o que acabamos de ver

1 Um ponto tem resíduo −0,3. Ele está acima ou abaixo da reta?

2 Por que não basta somar os resíduos?

3 Com $\beta_1 = 1{,}9$ o custo é 2,9, igual ao de $\beta_1 = 1{,}3$. Por quê?

Quatro casas, quatro equações

$$2 = \beta_0 \cdot 1 + \beta_1 \cdot 1 + e_1$$

$$4 = \beta_0 \cdot 1 + \beta_1 \cdot 2 + e_2$$

$$5 = \beta_0 \cdot 1 + \beta_1 \cdot 3 + e_3$$

$$7 = \beta_0 \cdot 1 + \beta_1 \cdot 4 + e_4$$

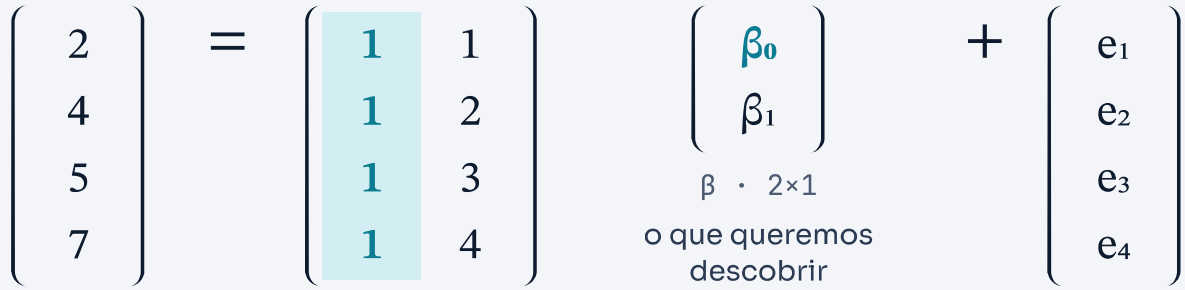
β_0 e β_1 são os **mesmos** em todas as linhas. Só mudam os dados de cada casa.

β_0 sempre aparece multiplicado por **1**. Guarde isso: é o motivo da coluna de 1s.

Os valores em **laranja** são as áreas: a parte que muda de casa para casa.

CONCLUSÃO Com 20 640 casas e 8 características, escrever equação por equação é inviável. A notação matricial guarda todas de uma vez: $y = X\beta + e$

y, X e β: quem é quem



y · 4×1
preços reais

X · 4×2
coluna de 1s + área

β · 2×1
o que queremos descobrir

e · 4×1
resíduos

Linha = uma casa

Cada linha de X e de y descreve um exemplo.

Coluna = uma característica

Com p características, X tem p + 1 colunas.

Coluna de 1s = intercepto

Sem ela, $\hat{y} = \beta_1 x$ e a reta passa à força pela origem: casa de 0 m² valendo R$ 0.

Xβ calcula todas as previsões de uma vez

$$\underbrace{\begin{pmatrix} 1 & 1 \\ 1 & 2 \\ 1 & 3 \\ 1 & 4 \end{pmatrix}}_{X \cdot 4\times 2} \cdot \underbrace{\begin{pmatrix} 0{,}5 \\ 1{,}6 \end{pmatrix}}_{\beta \cdot 2\times 1} = \underbrace{\begin{pmatrix} 2{,}1 \\ 3{,}7 \\ 5{,}3 \\ 6{,}9 \end{pmatrix}}_{\hat{y} \cdot 4\times 1}$$

linha 3 $1 \cdot 0{,}5 + 3 \cdot 1{,}6 = 0{,}5 + 4{,}8 = 5{,}3$

REGRA Cada linha de X é multiplicada elemento a elemento por β, e os produtos são somados. Uma linha, uma previsão.

Dimensões precisam combinar

$$(n \times \mathbf{k}) \cdot (\mathbf{k} \times m) = (n \times m)$$

Os números de dentro precisam ser iguais. Os de fora dão o tamanho do resultado.

operação	formatos	resultado
$X\beta$	(4×2)(2×1)	4×1 ✓
X^T	—	2×4
X^TX	(2×4)(4×2)	2×2 ✓
X^Ty	(2×4)(4×1)	2×1 ✓
X·X	(4×2)(4×2)	erro ✗

TRANSPOSTA

$$\begin{pmatrix} \mathbf{1} & 1 \\ \mathbf{1} & 2 \\ \mathbf{1} & 3 \\ \mathbf{1} & 4 \end{pmatrix} \longrightarrow \begin{pmatrix} \mathbf{1} & \mathbf{1} & \mathbf{1} & \mathbf{1} \\ 1 & 2 & 3 & 4 \end{pmatrix}$$

X · 4×2 — X^T · 2×4

Linhas viram colunas. Em NumPy: `X.T`

X^TX e X^Ty sempre têm p + 1 linhas, não importa quantas casas existam.

EXERCÍCIO GUIADO 1 · 5 MIN

Monte as matrizes você mesmo

casa	área (m^2)	preço (R$ mil)
A	120	300
B	80	210
C	150	360

Use a área em m^2, sem converter.

01 Escreva o vetor y.

02 Escreva X com a coluna de 1s.

03 Quais são as dimensões de X, β, X^T e X^TX?

04 X @ y é uma operação válida? E X^T @ y?

O custo em linguagem de matrizes

$e = y - X\beta$ vetor de resíduos, um por casa

$$\underset{e^{T}\ \cdot\ 1\times4}{\begin{pmatrix} 0 & 1 & 1 & 2 \end{pmatrix}} \cdot \underset{e\ \cdot\ 4\times1}{\begin{pmatrix} 0 \\ 1 \\ 1 \\ 2 \end{pmatrix}} = 0\cdot0 + 1\cdot1 + 1\cdot1 + 2\cdot2 = \mathbf{6}$$

É o mesmo 6 da reta ŷ = 1 + x no bloco A. $e^{T}e$ é a soma dos quadrados dos resíduos, também escrita $\|e\|^2$.

OBJETIVO Objetivo: encontrar o β que torna $J(\beta) = (y - X\beta)^{T}(y - X\beta)$ o menor possível.

A dedução em cinco passos

1	**Escrever o custo**	$J(\beta) = (y - X\beta)^T(y - X\beta)$	soma dos erros ao quadrado
↓			
2	**Expandir**	$y^Ty - 2\beta^TX^Ty + \beta^TX^TX\beta$	como $(a - b)^2 = a^2 - 2ab + b^2$
↓			
3	**Derivar**	$\nabla J = -2X^Ty + 2X^TX\beta$	a inclinação em cada direção
↓			
4	**Igualar a zero**	$X^TX\beta = X^Ty$	o fundo da tigela: as equações normais
↓			
5	**Isolar β**	$\hat{\beta} = (X^TX)^{-1}X^Ty$	multiplicar pela inversa

Passo 2 · Expandir o produto

com números $(a-b)^2 = a^2 - 2ab + b^2$

com matrizes $(y - X\beta)^T(y - X\beta) = y^Ty - y^TX\beta - \beta^TX^Ty + \beta^TX^TX\beta$

REGRA DA TRANSPOSTA

$(AB)^T = B^TA^T$, então $(y - X\beta)^T = y^T - \beta^TX^T$

OS DOIS TERMOS DO MEIO

$y^TX\beta$ e β^TX^Ty são números (1×1), e um é a transposta do outro. Logo são iguais e somam $-2\beta^TX^Ty$.

$$J(\beta) = \underbrace{y^Ty}_{\text{não depende de } \beta} \quad \underbrace{- 2\beta^TX^Ty}_{\text{linear em } \beta} \quad \underbrace{+ \beta^TX^TX\beta}_{\text{quadrático em } \beta}$$

Passos 3 e 4 · Derivar e igualar a zero

	função	derivada
uma variável	$J(b) = c - 2ab + kb^2$	$J'(b) = -2a + 2kb$
matrizes	$J(\beta) = y^T y - 2\beta^T X^T y + \beta^T X^T X\beta$	$\nabla J = -2X^T y + 2X^T X\beta$

O gradiente ∇J é a lista das derivadas de J, uma para cada coeficiente. No mínimo, todas valem zero.

IGUALAR A ZERO

$$-2X^T y + 2X^T X\beta = 0$$

$$X^T X\beta = X^T y$$

as equações normais

CONFERINDO NO EXEMPLO

$J(\beta_1) = 77 - 96\beta_1 + 30\beta_1^2$

$J' = -96 + 60\beta_1 = 0 \Rightarrow \beta_1 = 1{,}6$

o fundo da parábola do bloco A

Passo 5 · Isolar β com a inversa

números $5x = 10 \Rightarrow x = 5^{-1} \cdot 10 = 2$

matrizes $A^{-1}A = I \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}$ I é o "1" das matrizes

$$X^T X\,\beta = X^T y$$

$$(X^T X)^{-1} X^T X\,\beta = (X^T X)^{-1} X^T y$$

$$\hat{\beta} = (X^T X)^{-1} X^T y$$

QUANDO A INVERSA EXISTE?

A matriz precisa ser quadrada. $X^T X$ sempre é: (p+1)×(p+1).

E ter determinante ≠ 0, o que acontece quando as colunas de X são linearmente independentes.

INVERSA 2×2

$$\begin{pmatrix} a & b \\ c & d \end{pmatrix}^{-1} = 1/(ad - bc) \begin{pmatrix} d & -b \\ -c & a \end{pmatrix}$$

Se ad − bc = 0, dividimos por zero: não há inversa.

O que cada pedaço da fórmula faz

$$\hat{\beta} = (X^T X)^{-1} X^T y$$

$X^T y$

(p+1)×1

Para cada coluna de X, soma coluna × preço. Mede o quanto cada característica anda junto com y.

$X^T X$

(p+1)×(p+1)

Soma coluna × coluna para cada par. Mede o quanto as características se sobrepõem.

$(X^T X)^{-1}$

(p+1)×(p+1)

Desconta a sobreposição e separa o efeito de cada coluna. O resultado β̂ tem um número por coluna de X.

Exemplo resolvido com as quatro casas

1 · X^TX

$$\begin{pmatrix} 4 & 10 \\ 10 & 30 \end{pmatrix}$$

n, Σx
Σx, Σx²

2 · X^Ty

$$\begin{pmatrix} 18 \\ 53 \end{pmatrix}$$

Σy
Σxy = 2 + 8 + 15 + 28

3 · $(X^TX)^{-1}$

$$det = 4 \cdot 30 - 10 \cdot 10 = 20$$

$$\begin{pmatrix} 1{,}5 & -0{,}5 \\ -0{,}5 & 0{,}2 \end{pmatrix}$$

= (1/20)·[30 −10; −10 4]

4 · $\hat{\beta}$

$$1{,}5 \cdot 18 - 0{,}5 \cdot 53 = 0{,}5$$

$$-0{,}5 \cdot 18 + 0{,}2 \cdot 53 = 1{,}6$$

$$\begin{pmatrix} \mathbf{0{,}5} \\ \mathbf{1{,}6} \end{pmatrix}$$

CONCLUSÃO $\beta_0 = 0{,}5$ e $\beta_1 = 1{,}6$: a reta começa em R$ 50 mil e cada 100 m² a mais somam R$ 160 mil. É a reta do início da aula.

INTERPRETAÇÃO GEOMÉTRICA

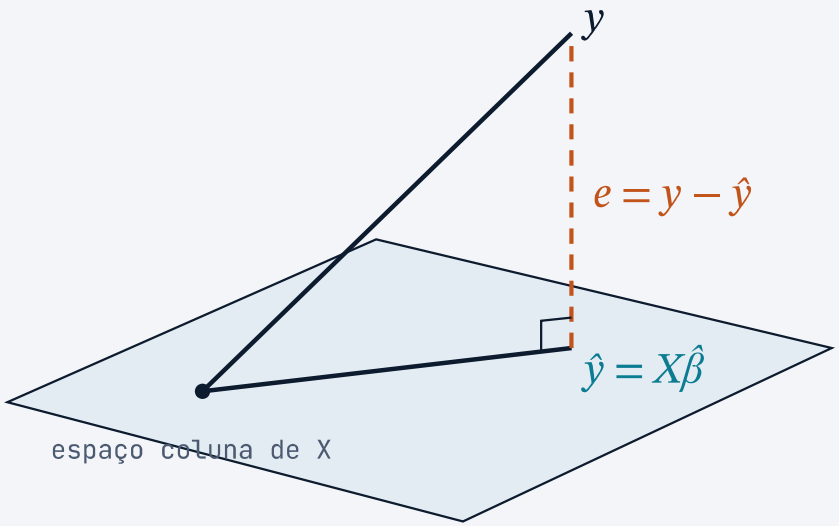
A previsão é a sombra de y

Todo Xβ possível mora no plano gerado pelas colunas de X. A melhor previsão ŷ é a projeção de y nesse plano, e o resíduo fica em ângulo reto com ele.

ÂNGULO RETO COM AS COLUNAS

$$X^T e = 0 \Leftrightarrow X^T X\beta = X^T y$$

coluna de X	Σ coluna · e
1s	−0,1 + 0,3 − 0,3 + 0,1 = 0
x	−0,1 + 0,6 − 0,9 + 0,4 = 0



Quando duas colunas carregam a mesma informação

$$\begin{pmatrix} 1 & 1 & 1 \\ 1 & 2 & 2 \\ 1 & 3 & 3 \\ 1 & 4 & 4 \end{pmatrix} \longrightarrow \begin{pmatrix} 4 & 10 & 10 \\ 10 & 30 & 30 \\ 10 & 30 & 30 \end{pmatrix}$$

X com x repetida

X^TX · det = 0

DEFINIÇÃO

Colunas linearmente independentes: nenhuma coluna pode ser escrita como combinação das outras.

INFINITAS RESPOSTAS, MESMO ERRO

β_1	β_2	$\hat{y}$
1,6	0	2,1 3,7 5,3 6,9
0,8	0,8	2,1 3,7 5,3 6,9
3,6	−2	2,1 3,7 5,3 6,9
101,6	−100	2,1 3,7 5,3 6,9

Com $\beta_0 = 0{,}5$, qualquer par com $\beta_1 + \beta_2 = 1{,}6$ dá as mesmas previsões.

Na prática: área em m² e em pés²; todas as dummies de uma categoria junto com a coluna de 1s.

Três perguntas diferentes

Existe solução?

Sempre ✓

Sempre há pelo menos um β que minimiza J: a projeção de y no plano sempre existe.

É única?

Depende de X

Só quando as colunas de X são independentes. Isso exige n ≥ p + 1.

É estável?

Depende do condicionamento

Se X é quase colinear, mudanças pequenas nos dados mudam muito o β. Medida: np.linalg.cond.

CONCLUSÃO Formar X^TX eleva o problema ao quadrado: $cond(X^TX) = cond(X)^2$. inv e solve exigem unicidade; pinv responde mesmo sem ela.

PERGUNTA RÁPIDA · 3 MIN

Antes de ir para o computador

1 O que acontece com a reta se X não tiver a coluna de 1s?

2 Duas colunas de X são idênticas. Quantas soluções de mínimos quadrados existem?

3 Por que X^TX é sempre quadrada, não importa quantas casas existam?

Três jeitos de calcular β no NumPy

Inversa

```
np.linalg.inv(X.T @ X) @ X.T @ y
```

COMO FUNCIONA

Monta $(X^TX)^{-1}$ inteira e multiplica. A fórmula do quadro, literal.

COM X^TX SINGULAR

LinAlgError, ou números enormes sem aviso se for quase singular.

ESTABILIDADE

A menor: forma X^TX e ainda calcula a inversa.

QUANDO USAR

Para estudar e conferir a fórmula.

Solve

```
np.linalg.solve(X.T @ X, X.T @ y)
```

COMO FUNCIONA

Resolve $X^TX\beta = X^Ty$ por eliminação, sem montar a inversa.

COM X^TX SINGULAR

LinAlgError. Evitar a inversa não cria uma solução única.

ESTABILIDADE

Melhor que inv, mas ainda forma X^TX.

QUANDO USAR

Colunas independentes e bem escaladas.

Pseudoinversa

```
np.linalg.pinv(X) @ y
```

COMO FUNCIONA

Decompõe X pela SVD e inverte só as direções que existem.

COM X^TX SINGULAR

Funciona: devolve a solução de menor norma $\|\beta\|$.

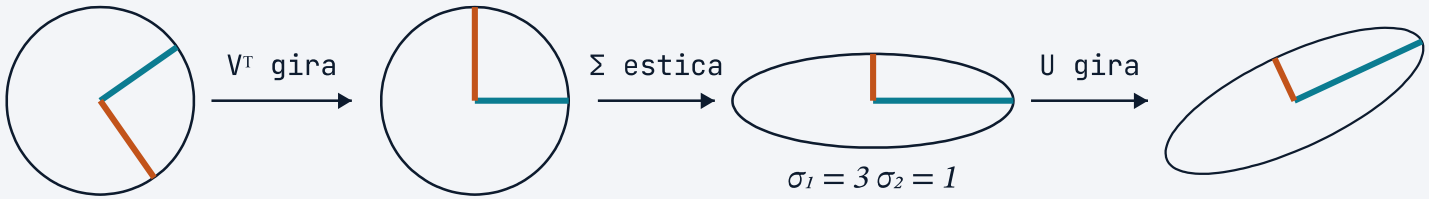
ESTABILIDADE

A maior: trabalha com X, sem formar X^TX.

QUANDO USAR

Padrão seguro. Custo um pouco maior.

A ideia da SVD: girar, esticar, girar



$$X = U \Sigma V^T$$

Toda matriz pode ser escrita assim. A diagonal de Σ guarda os valores singulares σ: quanto cada direção é esticada. Um σ = 0 indica uma direção sem informação, ou seja, colinearidade.

$$X^+ = V \Sigma^+ U^T \cdot \hat{\beta} = X^+ y$$

Σ⁺ troca cada σ por 1/σ, e coloca 0 onde σ ≈ 0. Exemplo: diag(5, 2, 0) vira diag(0,2; 0,5; 0).

Resultado: a solução de menor norma. Se X^TX for invertível, $X^+y = (X^TX)^{-1}X^Ty$.

TENTE PREVER ANTES DE EXECUTAR

O que cada linha devolve?

```
x = np.array([1., 2., 3., 4.])
y = np.array([2., 4., 5., 7.])
X_dup = np.column_stack([np.ones(4), x, x])

A, b = X_dup.T @ X_dup, X_dup.T @ y
np.linalg.inv(A) @ b        # ?
np.linalg.solve(A, b)       # ?
np.linalg.pinv(X_dup) @ y   # ?
```

X^TX tem inversa?

Quantas soluções existem?

Entre elas, qual tem a menor norma?

O resultado

inv

```
LinAlgError:
Singular matrix
```

Não há inversa para calcular.

solve

```
LinAlgError:
Singular matrix
```

Sem solução única, não há o que resolver.

pinv

```
array([0.5, 0.8, 0.8])
```

Divide o 1,6 igualmente: o par (0,8; 0,8) tem a menor norma entre todos com $\beta_1 + \beta_2 = 1{,}6$.

ATENÇÃO

Aqui a cópia é exata e o erro aparece. Se a coluna redundante tiver arredondamento (área em pés² = 10,764 × m²), inv pode devolver números enormes **sem erro nenhum**. Confira sempre `np.linalg.cond` .

Passo 1 · Carregar os dados e conferir formatos

```python
import numpy as np
from sklearn.datasets import fetch_california_housing

dados = fetch_california_housing()
X, y = dados.data, dados.target

print(X.shape, y.shape)      # (20640, 8) (20640,)
print(dados.feature_names)   # ['MedInc', 'HouseAge', ...]
```

SEM INTERNET? USE AS QUATRO CASAS

```python
X = np.array([1., 2., 3., 4.]).reshape(-1, 1)  # (4, 1)
y = np.array([2., 4., 5., 7.])                 # (4,)
```

	o que é
`X`	20 640 regiões da Califórnia × 8 características
`y`	valor mediano das casas, em US$ 100 mil
`(20640,)`	vetor 1D: uma posição por região

A primeira execução baixa o dataset e guarda em cache.

Passo 2 · Adicionar a coluna de 1s

```
n = X.shape[0]
Xb = np.column_stack([np.ones(n), X])

print(Xb.shape)      # (20640, 9)
print(Xb[0, :3])     # 1.0, 8.3252, 41.0
```

Cada linha agora começa com 1, e β terá 9 posições: β_0 mais um coeficiente por característica.

expressão	formato	o que é
`Xb`	(20640, 9)	dados + coluna de 1s
`Xb.T`	(9, 20640)	transposta
`Xb.T @ Xb`	(9, 9)	sobreposição entre colunas
`Xb.T @ y`	(9,)	coluna × preço
`beta`	(9,)	β_0 + 8 coeficientes
`Xb @ beta`	(20640,)	uma previsão por região

Passo 3 · Calcular β pelos três métodos

```
A = Xb.T @ Xb          # (9, 9)
b = Xb.T @ y           # (9,)

beta_inv   = np.linalg.inv(A) @ b
beta_solve = np.linalg.solve(A, b)
beta_pinv  = np.linalg.pinv(Xb) @ y

print(np.allclose(beta_solve, beta_pinv))   # ?
print(np.abs(beta_inv - beta_pinv).max())

y_hat = Xb @ beta_pinv   # (20640,)
```

TENTE PREVER

O allclose vai dar True ou False? Por quê?

`A` e `b` são os dois lados de $X^TX\beta = X^Ty$.

`pinv` não precisa de A: usa Xb direto.

`y_hat` tem uma previsão por região.

Passo 4 · O que os coeficientes dizem

valores esperados, arredondados · rode e confira

coluna	β	leitura
intercepto	−36,94	sem sentido físico: latitude e longitude 0
MedInc	0,4367	+US$ 10 mil de renda → +US$ 44 mil
HouseAge	0,0094	+1 ano de idade → +US$ 940
AveRooms	−0,1073	negativo: sobreposição com AveBedrms
AveBedrms	0,6451	
Population	−0,000004	pequeno porque a escala é grande
AveOccup	−0,0038	
Latitude	−0,4213	mais ao norte → menor valor
Longitude	−0,4345	mais a leste → menor valor

MANTENDO O RESTO FIXO

Cada coeficiente mede o efeito de uma coluna com todas as outras paradas.

TAMANHO NÃO É IMPORTÂNCIA

O valor depende da escala da coluna. Population é medida em pessoas; MedInc, em dezenas de milhares de dólares.

R^2: quanto do erro o modelo eliminou

$$R^2 = 1 - SQres / SQtot$$

	fórmula	é o erro de
SQres	$\Sigma(y_i - \hat{y}_i)^2$	o modelo
SQtot	$\Sigma(y_i - \bar{y})^2$	chutar sempre a média

SQtot é o custo da reta horizontal do bloco A.

QUATRO CASAS

SQres = 0,2 · SQtot = 13

$$R^2 = 1 - 0{,}2 / 13 \approx 0{,}985$$

< 0	0	1
pior que a média	igual à média	perfeito

```
r2 = 1 - np.sum((y - y_hat)**2) / np.sum((y - y.mean())**2)
```

Passo 5 · Conferir com o LinearRegression

```
from sklearn.linear_model import LinearRegression

lr = LinearRegression()    # fit_intercept=True
lr.fit(X, y)               # X SEM a coluna de 1s
beta_sk = np.r_[lr.intercept_, lr.coef_]

print(np.allclose(beta_pinv, beta_sk))   # True
print(lr.score(X, y))                    # ≈ 0.606
```

`intercept_` guarda β_0; `coef_` guarda os outros 8.

`np.r_` junta os dois em um vetor de 9, na mesma ordem do nosso.

`score` devolve o R^2. Cerca de 0,61: o modelo explica 61% da variação nos dados de treino.

Por dentro, usa um resolvedor de mínimos quadrados (scipy.linalg.lstsq), não a inversa.

Erros comuns · parte 1

01 Esquecer a coluna de 1s

Roda sem erro, mas a reta passa à força pela origem e os outros coeficientes compensam.

CORREÇÃO np.column_stack([np.ones(n), X])

02 Formatos incompatíveis

X (4, 2) vezes X (4, 2): os números de dentro não batem.

```
X @ X   # ValueError: matmul ... mismatch
```

CORREÇÃO confira .shape antes; use X.T @ X

03 * no lugar de @

Não dá erro: multiplica elemento a elemento e devolve (4, 2) em vez de 4 previsões.

```
X * beta   # (4, 2)  X
```

CORREÇÃO X @ beta # (4,) ✓

04 Matriz singular

Colunas repetidas ou proporcionais, ou todas as dummies junto com a coluna de 1s.

```
np.linalg.inv(A)   # LinAlgError
```

CORREÇÃO remova a coluna redundante (drop_first=True) ou use pinv

Erros comuns · parte 2

05 Inversa sem necessidade

Mais contas, mais arredondamento, e a menor estabilidade dos três métodos.

```
inv(A) @ b
```

CORREÇÃO np.linalg.solve(A, b) ou np.linalg.pinv(Xb) @ y

06 Escalas muito diferentes

Population chega a dezenas de milhares; AveBedrms fica perto de 1. O número de condição cresce e β fica sensível.

```
np.linalg.cond(Xb)
```

CORREÇÃO padronize com média e desvio do treino; os coeficientes passam a ser “por desvio-padrão”

07 Intercepto em dobro

Passar Xb (com 1s) ao LinearRegression: coef_[0] vira ≈ 0 e β_0 vai para intercept_.

```
LinearRegression().fit(Xb, y)
```

CORREÇÃO passe X sem 1s, ou use fit_intercept=False com Xb

Treinar, prever, avaliar

`fit(X, y)`	→	adiciona coluna de 1s	→	resolve com inv, solve ou pinv	→	guarda self.coef_
`predict(X)`	→	adiciona coluna de 1s	→	Xb @ self.coef_	→	devolve ŷ
`score(X, y)`	→	ŷ = self.predict(X)	→	calcula SQres e SQtot	→	devolve R^2

TREINO × PREDIÇÃO

fit aprende β com dados que têm resposta. predict usa β em dados novos, sem resposta.

PERGUNTA

Por que predict precisa adicionar a coluna de 1s de novo, se o fit já fez isso?

EXERCÍCIO GUIADO 2 · 8 MIN

Aquecimento com as quatro casas

```
x = np.array([1., 2., 3., 4.])
y = np.array([2., 4., 5., 7.])
Xb = np.column_stack([np.ones(4), x])

beta = ...       # 1
y_novo = ...     # 2  casa de 250 m²
r2 = ...         # 3
```

01 Calcule β com np.linalg.solve.

02 Preveja o preço de uma casa de 250 m². Que linha nova de X você precisa montar?

03 Calcule o R^2 do ajuste.

04 Em uma frase: o que significa β_1?

DESAFIO FINAL

A classe MinimosQuadrados

`MinimosQuadrados(metodo="pinv")`	aceita "inv", "solve" ou "pinv"; qualquer outro valor gera ValueError
`fit(X, y)`	adiciona a coluna de 1s, calcula β, guarda em self.coef_ e devolve self
`predict(X)`	aplica o mesmo tratamento do fit e devolve $\hat{y}$
`score(X, y)`	devolve o R^2
`regras`	só NumPy; nenhum for percorrendo linhas ou colunas

ENTREGA Entrega: notebook com a classe, os testes do slide de validação e uma célula com o caso da coluna duplicada.

Estrutura inicial

```python
import numpy as np

class MinimosQuadrados:
    def __init__(self, metodo="pinv"):
        # TODO: validar metodo e guardar em self.metodo

    def _adicionar_intercepto(self, X):
        # TODO: devolver X com uma coluna de 1s à esquerda

    def fit(self, X, y):
        # TODO: montar Xb e calcular self.coef_
        return self

    def predict(self, X):
        # TODO: mesmo tratamento do fit, depois Xb @ self.coef_

    def score(self, X, y):
        # TODO: R² = 1 - SQres / SQtot
```

ONDE VOCÊ JÁ VIU

parte	slide
`_adicionar_intercepto`	30
`fit · inv/solve/pinv`	25, 31
`predict`	11, 37
`score`	33

Etapas, cada uma com um ponto de controle

etapa	implemente	ponto de controle
1	`__init__`	MinimosQuadrados("abc") gera ValueError
2	`_adicionar_intercepto`	X (4, 1) vira (4, 2), primeira coluna só 1s
3	`fit` com "solve"	quatro casas → coef_ ≈ [0.5, 1.6]
4	`fit` com "inv" e "pinv"	os três métodos dão o mesmo coef_
5	`predict`	quatro casas → [2.1, 3.7, 5.3, 6.9]
6	`score`	quatro casas → ≈ 0.985
7	California Housing	validação contra o LinearRegression

Como validar a sua classe

```
m  = MinimosQuadrados(metodo="pinv").fit(X, y)
lr = LinearRegression().fit(X, y)

np.allclose(m.coef_, np.r_[lr.intercept_, lr.coef_])
np.allclose(m.predict(X), lr.predict(X))
np.isclose(m.score(X, y), lr.score(X, y))

X_dup = np.column_stack([X, X[:, 0]])   # MedInc repetida
np.linalg.cond(m._adicionar_intercepto(X))
np.linalg.cond(m._adicionar_intercepto(X_dup))
MinimosQuadrados(metodo="inv").fit(X_dup, y)   # ?
```

CRITÉRIOS DE ACEITE

01 Os três allclose dão True para os três métodos.

02 Com X_dup, o notebook mostra o que inv e solve fazem e que pinv ainda funciona.

03 O número de condição é impresso nos dois casos.

04 Nenhum for sobre linhas ou colunas dentro da classe.

O que levar desta aula

01 Mínimos quadrados escolhe o β que minimiza a soma dos resíduos ao quadrado.

02 Igualar o gradiente a zero dá as equações normais: $X^TX\beta = X^Ty$.

03 inv e solve precisam de colunas independentes. pinv sempre responde, com a solução de menor norma.

04 No código: coluna de 1s, @ em vez de *, e conferir .shape.

LEITURA

numpy.linalg.lstsq · numpy.linalg.pinv · numpy.linalg.cond · scikit-learn: Linear Models

PRÓXIMA AULA · 6.3

Gradiente descendente: o mesmo β, chegando ao fundo da tigela passo a passo.